

Criptoanálisis Distribuido y Hijacking de Rutas RIP

Integración de seguridad informática en redes y sistemas de software (Gpo 403)
TC2007B

Santiago Hernández
A01787550

September 22, 2026

1 Introduction

[Main idea: motivation and threat model for attacking RIPv2 authentication. Then, in order: RIPv2 distributes routing updates with only optional authentication, and the RFC 2082 Baker and Atkinson (1997) keyed message-digest mode is the sole barrier to a forged update; an on-path attacker who forges an accepted update can inject or override routes to blackhole or man-in-the-middle traffic; this work stays honest to the lab scope by doing offline key recovery from the provided captures, not a live attack. Close on: the recovered key is what would unlock a forged update, with live injection left as a documented stub (`inject_route.py`).]

[Main idea: what this report does and the roadmap through the sections. Then, in order: reverse-engineer the exact digest construction from a capture (Section 2); build a master–worker cluster that partitions the keyspace across TCP-connected workers (Section 3, Section 4); tolerate node failure so no chunk is left unprocessed (Section 5); characterize throughput (Section 6). Close on: the captures use six-character base-36 keys, recoverable in minutes at the rates we measure.]

2 Reverse-Engineering the RIP MD5 Authentication

Packet layout. [Main idea: each captured update is an authenticated RIPv2 Response, decomposing into RIP header, auth header, route entry, and auth trailer. Then, in order: each update is a RIPv2 Response (command 2) sent to multicast 224.0.0.9 on UDP/520; authentication type 3 (Keyed Message Digest) places an authentication header as the first route-table entry and appends an authentication trailer carrying the digest; the observed size-06 packet decomposes as in Table 1. Close on: the trailer's 16-byte digest is the field the attack must reproduce.]

Region	Bytes	Contents (size-06 capture)
RIP header	4	command=2, version=2, routing-domain=0
Auth header (RTE)	20	AFI=0xFFFF, auth-type=3, pkt-len=44, key-id=1, auth-len=20, seq, reserved
Route entry	20	the advertised route
Auth trailer	20	AFI=0xFFFF, type=0x0001, then the 16-byte digest

Table 1: RIPv2/RFC 2082 authenticated packet layout (64-byte payload).

Digest construction (the key finding). [Main idea: the captured digest is RFC 2082 keyed-MD5, not the HMAC-MD5 the brief names. Then, in order: the 16-byte digest field is first overwritten with the key and a plain MD5 is taken over the whole packet; define D as the payload bytes from the RIP header through the trailer's `AFI/type` field, i.e. everything except the 16 digest bytes, and K_{16} as the ASCII key right-padded with zero bytes to 16, as used in (2.1). Close on: this construction and notation must be stated before the equation.]

$$\text{digest} \;=\; \text{MD5}\big(\, D \,\|\, K_{16} \,\big). \tag{2.1}$$

[Main idea: how the construction and alphabet were confirmed by brute force. Then, in order: we tested both HMAC and keyed-MD5 and both candidate alphabets; only (2.1) over a base-36 alphabet (lowercase letters and digits, not hexadecimal) reproduced the captured digest; the hexadecimal space was exhausted with no match. Close on: base-36 keyed-MD5 is the confirmed construction.]

Recovered key and validation. [Main idea: the recovered key and that validation spans every packet, not one coincidence. Then, in order: the single-threaded oracle recovers `ykusrq` at index 1,618,816,210 of 36^6; the same key reproduces the digest of all 21 authenticated packets, each with a distinct sequence number and payload; Table 2 summarizes the corrected parameters against the brief. Close on: 21/21 validation rules out a single-packet coincidence.]

Parameter	Brief says	Actual (measured)	Evidence
Construction	HMAC-MD5	keyed-MD5 (RFC 2082)	only (2.1) matches
Alphabet	hex / "letters+numbers"	base-36	hex space exhausted, no match
Key length	6	6	filename `size_06` + match
Key (06_01)	—	`ykusrq`	21/21 packets validate

Table 2: Reverse-engineered authentication parameters.

Security implications. [Main idea: what the base-36 short-key finding means for real deployments. Then, in order: keyed-MD5 versus HMAC changes how a candidate is computed, not how many exist, so the weakness is the size of the space, not the MAC choice; the space is $36^6 = 2{,}176{,}782{,}336$ candidates, fully enumerable; at the rates in Section 6 that space falls in minutes, so short keys over a small alphabet are the exploitable weakness. Close on: an attacker who captures one authenticated packet can recover a short key offline, so the digest offers little protection at this key length and alphabet.]

3 TCP Coordination Protocol and Message Types

[Main idea: master and workers coordinate over raw TCP with a line-oriented protocol and pull-based work handout. Then, in order: each message is one JSON object terminated by a newline, so framing is trivial and the traffic is human-readable during debugging; the master never blocks a fast worker because chunks are handed out on request, so faster nodes request more often; Table 3 lists the message types. Close on: newline-delimited JSON over raw TCP was chosen for trivial framing rather than a length-prefixed or heavier RPC scheme.]

Message	Direction	Purpose / fields
`HELLO`	W→M	join; worker id, host, pid, cores
`PARAMS`	M→W	target data, digest, alphabet, length, construction
`REQUEST`	W→M	ask for a keyspace chunk
`ASSIGN`	M→W	{chunk id, start, end}
`PROGRESS`	W→M	{chunk, tried, rate}; doubles as a heartbeat
`HEARTBEAT`	W→M	liveness while idle between chunks
`DONE`	W→M	chunk exhausted with no match
`FOUND`	W→M	{chunk, key, index} on a match
`NOWORK`	M→W	queue momentarily empty; retry
`STOP`	M→W	global halt (key found, or keyspace exhausted)

Table 3: Master–worker message protocol.

Lifecycle. [Main idea: the per-worker message lifecycle and how a found key halts the cluster. Then, in order: a worker sends `HELLO`, receives `PARAMS`, then loops `REQUEST`→`ASSIGN`, cracking each chunk in sub-batches while emitting `PROGRESS`, and reports `DONE` or `FOUND`; on `FOUND` the master re-verifies the claimed key against the captured digest before trusting it, then broadcasts `STOP` so no node keeps hashing; the master is never idle, launching local worker subprocesses that connect back over loopback and crack alongside the remote workers. Close on: re-verification plus `STOP` guarantees a single trusted result and an immediate global halt.]

4 Keyspace Partition Strategy

[Main idea: the keyspace is indexed and split into fixed-size chunks served from a queue. Then, in order: the search space is every length-6 string over the base-36 alphabet, of size $N = 36^6 = 2{,}176{,}782{,}336$; each integer index $i \in [0, N)$ maps to a key by little-endian positional encoding, so a contiguous index interval is a set of distinct keys with no overlap between intervals; the master divides $[0, N)$ into fixed-size chunks of C keys, giving $\lceil N/C \rceil$ chunks (e.g. $C = 5 \times 10^6$ yields 436 chunks) served from a queue. Close on: contiguous

non-overlapping intervals are what make dynamic assignment and safe re-queue possible.]

Algorithm 1 Dynamic, non-overlapping assignment

```
pending ← queue(0, 1, ..., ⌈N/C⌉ − 1);  inflight ← ∅
while a REQUEST arrives and not found do
  if pending ≠ ∅ then
    c ← pop(pending);  inflight[c] ← (worker, now)
    send ASSIGN(c, cC, min((c+1)C, N))
  else if inflight ≠ ∅ then
    send NOWORK {outstanding chunk may be requeued}
  else
    mark exhausted;  broadcast STOP
  end if
end while
```

[Main idea: why dynamic pull-based assignment beats a static split and how it composes with failure recovery. Then, in order: pull-based assignment keeps heterogeneous or contended workers balanced automatically rather than fixing shares up front; a chunk is only ever "done" when its `DONE` arrives; so a lost chunk can be safely re-queued (Section 5). Close on: assignment and recovery share one invariant, a chunk is complete only on `DONE`.]

5 Node-Death Detection and Reassignment

[Main idea: no assigned chunk may be silently lost, and the master defends against two failure modes by requeueing to the front of pending. Then, in order: the risk is that a lost chunk lets the search terminate reporting "not found" while skipping the key; hard crash or kill, where the TCP socket closes, the master's read returns EOF, and the disconnect handler requeues every chunk that worker held; freeze or network partition, where the socket stays open but `PROGRESS` heartbeats stop, and a monitor thread requeues any chunk whose last heartbeat is older than a timeout T (default 5 s) and marks the node dead. Close on: both modes requeue to the front of `pending` so a surviving worker picks the chunk up.]

Empirical check. [Main idea: a controlled fault-injection run confirms no chunk is lost. Then, in order: with three workers, freeze one with `SIGSTOP` and kill another with `SIGKILL`; the master emits the expected events shown below, a timeout requeueing chunk 19 and a disconnect requeueing chunk 40; both requeued chunks are subsequently completed by the surviving worker. Close on: coverage of the keyspace stayed total, so neither failure mode drops a chunk.]

```
TIMEOUT: worker freeze-w1 silent >4.0s, requeued chunk 19
worker kill-w2 disconnected; requeued chunks [40]
```

6 Benchmark Metrics

[Main idea: benchmark setup and units. Then, in order: all measurements target the size-06 digest and the keyed-MD5 construction; the host has 10 logical cores; rates are millions of keys hashed per second (Mkeys/s). Close on: these units apply to Table 4 and the appendix scaling sweep.]

Measurement	Aggregate	Per node/core
BM1 single-core baseline	2.55	2.55
BM2 scaling, 8 processes	10.67	1.33
BM2 scaling, 10 processes	10.95	1.09
BM3 distributed, 8 TCP workers	12.56	≈1.75

Table 4: Hashing throughput (Mkeys/s). Full sweep in `results/benchmarks.txt`.

CPU-bound vs. network-bound. [Main idea: the workload is overwhelmingly compute-bound, so coordination is not the bottleneck. Then, in order: each worker's wall time splits into hashing versus time blocked on the socket; across the distributed run aggregate compute time was 69.1 s against 0.03 s of socket wait, i.e. 99.96% of time spent hashing; throughput therefore scales with CPU cores until the host saturates, as in Table 4 where per-core rate falls once the process count exceeds the physical-core budget. Close on: the TCP coordination protocol is not the bottleneck, CPU is.]

End-to-end. [Main idea: end-to-end recovery time, single-threaded versus cluster. Then, in order: full recovery of `ykusrq` took 738 s single-threaded versus 142 s on the contended cluster; an uncontended ∼12.6 Mkeys/s cluster covers the whole 2.18×10^9 space in well under that. Close on: the six-character base-36 space is a minutes-scale target at cluster rates.]

7 Conclusion and Deferred Work

[Main idea: summary of what was recovered and how. Then, in order: we recovered a RIPv2/RFC 2082 authentication key by reverse-engineering the keyed-MD5 construction, correcting the brief on both the MAC and the alphabet; we validated the key against every authenticated packet in the capture; a pull-based master–worker cluster over TCP with heartbeat- and disconnect-based failure recovery reproduces the same key and is strongly CPU-bound. Close on: the exploit is offline key recovery, and the result reproduces across the whole capture.]

Deferred: live route injection. [Main idea: the offensive injection step is deferred by design. Then, in order: the final step is forging an authenticated RIPv2 update with the recovered key and injecting it into a live FRR/Mininet topology with Scapy; it is implemented as a documented stub (`src/inject_route.py`); it was intentionally not executed here because it requires the Mininet sandbox. Close on: the stub in `src/inject_route.py` documents the injection path.]

[Main idea: what a real deployment should do, grounded in the recovery cost. Then, in order: use longer keys over a larger alphabet, since recovery cost scales as the alphabet size raised to the key length; use per-neighbor keys with rotation to bound blast radius and key lifetime; migrate off RIP to a protocol with stronger authentication as the structural fix. Close on: these follow from the recovery cost we measured and are recommendations, not defenses we evaluated.]

References

Baker, F. and Atkinson, R. (1997). RIP-2 MD5 Authentication. Technical Report RFC 2082, Internet Engineering Task Force. `https://www.rfc-editor.org/rfc/rfc2082`.

Appendix A: Full Benchmark Scaling Sweep

[Main idea: the raw multi-process scaling sweep behind BM2. Then, in order: single host, 10 logical cores, base-36 keyed-MD5, raw multiprocessing with no sockets; the table below lists aggregate and per-core Mkeys/s for 1 to 10 processes; per-core rate falls once the process count exceeds the physical-core budget, from efficiency cores and scheduling contention. Close on: source data in `results/benchmarks.json`.]

Processes	**Aggregate (Mkeys/s)**	**Per-core (Mkeys/s)**
1	1.99	1.99
2	3.94	1.97
4	7.09	1.77
6	8.89	1.48
8	10.67	1.33
10	10.95	1.09

Table 5: BM2 scaling sweep (raw multiprocessing, no sockets).